

Graph Algorithms

3. P and NP

(first part)

Martin Milanič
martin.milanic@upr.si
UP FAMNIT

October 30, 2025



3. P and NP – Contents

1. Motivation
2. Decision problems
3. Classes P, NP, and co-NP, a quiz
4. NP-hard and NP-complete problems
5. Polynomial reductions, proving NP-completeness, two exercises
6. Polynomially equivalent problems, Cook reductions (next time)

Motivation:

In Chapter 2, we discussed a number of problems for which **efficient algorithms** are known:

- variants of SHORTEST PATH
- MINIMUM SPANNING TREE
- MAXIMUM FLOW
- MAXIMUM MATCHING

For many other problems, **no efficient algorithms** are known.

This applies to:

- the graph coloring problem
- the traveling salesman problem
- the maximum independent set problem

When faced with a new problem that we want to solve, a natural question arises:

How do we determine whether an efficient algorithm exists for the problem?

If we find such an algorithm (and prove its correctness and polynomial-time complexity), then of course we answered the question.

What if we don't manage to do this?

In this case, there is another natural option:
we can try to show that the problem is **NP-hard**.

In this chapter we will give the basic definitions and results regarding this second option.

Decision problems

Decision problem: a problem for which the set of input instances divides into two sets depending on whether the answer is YES or NO.

Examples:

BIPARTITENESS

Input: A graph $G = (V, E)$.

Question: Is G bipartite?

TOPOLOGICAL SORTING

Input: A digraph $D = (V, E)$.

Question: Does D have a topological sort?

Example:

2-SATISFIABILITY

Input: Boolean variables $x_1, \dots, x_n$,
clauses $C_1, \dots, C_m$ of length 2 over $x_1, \dots, x_n$
[clause = a disjunction of *literals*
(variables or their negations)]

Question: Is there a satisfying truth assignment?

Any optimization problem can be converted into a decision problem.

This can be done by introducing an additional input that represents a lower or upper bound for the optimal solution value (a lower bound for maximization problems and an upper bound for minimization problems).

Example:

For the following variant of the shortest path problem:

SHORTEST PATH

Input: A digraph $D = (V, E)$, a weight function $w : E \rightarrow \mathbb{R}^+$, two vertices s and t .

Task: Find a shortest path from vertex s to vertex t .

we get the following decision problem:

SHORTEST PATH (DECISION VERSION)

Input: A digraph $D = (V, E)$, a weight function $w : E \rightarrow \mathbb{R}^+$, two vertices s and t , a real number r .

Question: Is there an s, t -path P in the digraph D such that $w(P) \leq r$?

Classes P, NP, and co-NP

We define three classes of decision problems: **P**, **NP**, and **co-NP**.

- **P** is the set of decision problems that can be solved by a polynomial algorithm

Intuitively: **P** is the set of problems that can be solved efficiently.

The following problems are all in P:

- BIPARTITENESS
- TOPOLOGICAL SORTING
- 2-SATISFIABILITY

The decision versions of the following optimization problems are all in P:

- SHORTEST PATH
- MINIMUM SPANNING TREE
- MAXIMUM FLOW
- MAXIMUM MATCHING
- MINIMUM VERTEX COVER IN BIPARTITE GRAPHS

- **NP** is the set of decision problems with the following property:
If the answer is YES, then there exists a *certificate* that enables us to verify this fact in polynomial time.

Intuitively: **NP** is the set of problems for which we can quickly verify a positive answer if we are given a solution.

(Formally: **NP** is the set of languages recognizable by some Turing machine in polynomial time.)

An example problem in NP

HAMILTONIAN CYCLE

Input: A graph $G = (V, E)$.

Question: Is there a Hamiltonian cycle in the graph G ?

Hamiltonian cycle = cycle containing all vertices of the graph

If the answer is YES, then there is a certificate that allows us to check this fact in polynomial time:

- If the answer is YES, then the graph G has a Hamiltonian cycle. Any Hamiltonian cycle $C = (v_1, \dots, v_n)$ can be taken as the certificate).
- In polynomial time, we can verify that C is a Hamiltonian cycle in the graph G . We need to check that:
 - every two consecutive vertices on the cycle (and also the last and the first) are adjacent,
 - no vertex in the cycle is repeated,
 - C contains all vertices of the graph G .

The HAMILTONIAN CYCLE problem is therefore in NP.

What is a “certificate”? Anything that can help us verify that the answer to the question is indeed YES, for example, a set of vertices or edges we are looking for.

Be careful: All we need to justify that a problem is in NP is to *describe* the corresponding certificate. (We do not need to show anything about how to compute a certificate. It is only necessary to justify that with the help of the certificate, we can check in polynomial time whether the answer to the question is indeed affirmative.)

Examples:

- We are given a graph G . Does G contain a Hamiltonian cycle?
(Certificate: a Hamiltonian cycle.)
- We are given a graph G and an integer k . Does the graph G have an independent set of size at least k ?
(Certificate: an independent set of size at least k .)

- **co-NP** is the set of decision problems with the following property:
If the answer is NO then there exists a certificate that enables us to verify this fact in polynomial time.

Example problem in co-NP

PRIMES

Input: an integer $n > 1$
(written, e.g., in the decimal system)

Question: Is n a prime number?

If the answer is NO, then there is a *certificate* that allows us to check this fact in polynomial time:

- If the answer is NO, then n is not prime and there are two positive integers p and q such that $p, q \geq 2$ and $n = p \cdot q$.
Any such pair of integers forms a certificate.
- In polynomial time, we can check that these conditions are met.

The PRIMES problem is therefore in co-NP.

co-NP: the set of decision problems for which a **negative** answer can be efficiently checked if we have a “certificate”

Example: We are given a graph G .

Is it true that all maximal independent sets in G have the same cardinality?

*An independent set in a graph G is **maximal** if it is not contained in any larger independent set. Equivalently: it is a dominating set.*

(A certificate: two maximal independent sets S_1 and S_2 with $|S_1| \neq |S_2|$.)

SATISFIABILITY

SATISFIABILITY

Input: Boolean variables $x_1, \dots, x_n$,
clauses $C_1, \dots, C_m$ over $x_1, \dots, x_n$
[clause = a disjunction of *literals*
(variables or their negations)]

Question: Is there a satisfying truth assignment?

Example: Consider the following instance of SATISFIABILITY:

$$\{x_1, x_2, x_3\},$$

$$C_1 = x_1 \vee x_2 \vee x_3, \quad C_2 = x_1 \vee \overline{x_2} \vee \overline{x_3}, \quad C_3 = \overline{x_2} \vee x_3, \quad C_4 = \overline{x_1} \vee \overline{x_3}.$$

$$(\overline{x_i} \equiv \neg x_i)$$

Setting $x_1 = \top$, $x_2 = x_3 = \perp$ all the three clauses take value true.

SATISFIABILITY is in NP:

If the answer is YES, then every satisfiable truth assignment is a certificate of this fact.

We can verify in polynomial time whether this is indeed a satisfying assignment: we just have to evaluate all the clauses.

It is widely believed that SATISFIABILITY is not in P and not in co-NP, though nobody knows for sure. ▲

A quiz

For each of the following problems, determine whether it belongs to the class NP or not.

MINIMUM VERTEX COVER

Input: A graph $G = (V, E)$.

Task: Find a minimum vertex cover in G .

VERTEX COVER

Input: A graph $G = (V, E)$, integer k .

Question: Does the graph G have a vertex cover C such that $|C| \leq k$?

Relations between classes P, NP, and co-NP

Proposition

$$P \subseteq NP \cap co-NP.$$

Proof: $P \subseteq NP$: the certificate is empty.

We can verify a positive answer in polynomial time by solving the problem completely!

Similarly for $P \subseteq co-NP$.

NP-hard and NP-complete problems

There is a class of so-called NP-hard problems that most people *believe* are not solvable in polynomial time, but no one has ever proved that.

A problem is NP-**hard** if it is at least as hard as all problems in NP.

More precisely:

A problem Π is NP-**hard**,

if the existence of a polynomial-time algorithm for the problem Π

implies the existence of a polynomial-time algorithm for all problems in NP.

Equivalently:

Π is NP-hard $\Leftrightarrow$

If Π is solvable in polynomial time, then $P = NP$.

Intuitively:

if we could efficiently solve just one NP-hard problem Π , then we would be able to solve efficiently every problem whose solution we can verify quickly (by means of an algorithm for problem Π).

Note:

NP-hard problems are not necessarily decision problems!

They can also be optimization problems or search problems.

There is an important subclass of NP-hard problems – the class of NP-**complete** problems.

Definition

A problem is **NP-complete** if it is in NP and is NP-hard.

- Every NP-complete problem is a decision problem.
- These are the hardest problems in NP.

If there exists a polynomial-time algorithm for just one NP-complete problem, then all NP-complete problems are polynomially solvable.

- Thousands of NP-complete problems are known. A polynomial algorithm for any of them is very unlikely.

Note: Not every NP-hard decision problem is in NP. For example, this is the case for problems that are **undecidable** (that is, they cannot be solved by *any* algorithm).

Example: Given a finite set S of graphs, is there a graph G such that the neighborhood of each vertex in G induces a graph isomorphic to a graph in S ?

Three conjectures

There is a widely adopted conjecture that $P \neq NP$.

(Even if we can quickly check a positive answer, there is no reason to be able to quickly compute a solution.)

This is one of the most important open questions in mathematics and computer science.

The Clay Mathematics Institute offers **one million USD** for a correct solution.

P VERSUS NP A MILLION-DOLLAR QUESTION (AND MORE)

Clément Dallard

Izleti v matematično vesolje
Univerza na Primorskem

November 24, 2021



<https://matematicni-izleti.famnit.upr.si/sl/zgodovina/202122/program/p-versus-np/>

Another widely adopted conjecture is that $\text{NP} \neq \text{co-NP}$.

(Even if we can quickly check the positive answer, there is no reason why a negative answer can also be checked quickly.

For example, no one knows whether there is a short proof of the fact that a given conjunction of clauses is unsatisfiable.)

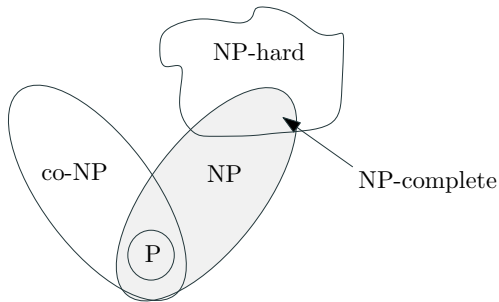
Conjecture: $\text{P} = \text{NP} \cap \text{co-NP}$.

This one is not so widely believed, though Jack Edmonds (one of the founders of combinatorial optimization) believes in the conjecture.



Source: <http://pretty.structures.free.fr/photos/photos6.html>
<http://pretty.structures.free.fr/photos/photos7.html>

The most likely relations between the classes NP, co-NP, NP-complete, and NP-hard problems:



The existence of NP-complete problems is not immediately evident.

Theorem (Cook 1971, Levin 1973)

SATISFIABILITY *is* NP-complete.

How do we show that a problem is NP-complete?

We'll make use of a concept that we've already seen in the second week of lectures:

Π_1, Π_2 decision problems

Definition

A **polynomial reduction** of problem Π_1 to problem Π_2 is a polynomial-time algorithm that for every instance I of Π_1

construct an instance $J = J(I)$ for problem Π_2 such that

the answer to Π_1 given I is the same as the answer to Π_2 given J .

If there exists a polynomial reduction from problem Π_1 to problem Π_2 , we denote this by $\Pi_1 \propto \Pi_2$.

Meaning: *"Problem Π_1 is not harder than problem Π_2 ."*

To show that a problem Π is NP-hard,
we reduce a known NP-hard problem to Π .

Proposition

Suppose that for a problem $\Pi \in \text{NP}$ there exists an NP-complete problem Π_1 such that $\Pi_1 \propto \Pi$.

Then Π is NP-complete.

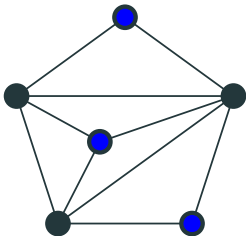
Examples of NP-complete Problems and Polynomial Reductions

INDEPENDENT SET

Input: A graph $G = (V, E)$, integer k .

Question: Does G contain an independent set of size $\geq k$?

independent set: a subset $S \subseteq V$ such that $u, v \in S \Rightarrow uv \notin E$



INDEPENDENT SET

Input: A graph $G = (V, E)$, integer k .

Question: Does G contain an independent set of size $\geq k$?

Proposition

The INDEPENDENT SET problem is NP-complete.

Proof:

1. INDEPENDENT SET $\in$ NP: we take an independent set $S \subseteq V$ with size at least k as a certificate.

We can verify in polynomial time whether $|S| \geq k$ and S is an independent set.

2. Reduction from SATISFIABILITY:

SATISFIABILITY $\propto$ INDEPENDENT SET:

Consider an instance I of SATISFIABILITY: $x_1, \dots, x_n, C_1, \dots, C_m$.

We will construct an instance $J = (G, k)$ of INDEPENDENT SET such that the disjunction of given clauses is satisfiable if and only if G has an independent set of size k .

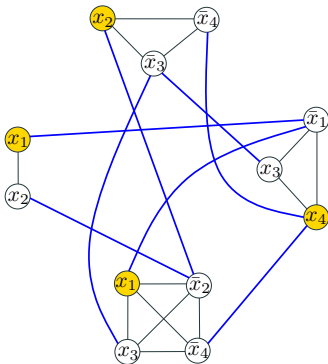
- Graph G has a vertex for each occurrence of every variable or its negation in the clauses.
- Two vertices are joined by an edge if and only if:
 - (1) they represent variables or their negations in the same clause, or
 - (2) they represent some variable and its negation.
- $k = m$.

Let us illustrate the reduction on an example.

The set of clauses

$$x_1 \vee x_2, \quad x_2 \vee \bar{x}_3 \vee \bar{x}_4, \quad \bar{x}_1 \vee x_3 \vee x_4, \quad x_1 \vee \bar{x}_2 \vee x_3 \vee \bar{x}_4$$

over the variables x_1, x_2, x_3, x_4 gets mapped to $(G, 4)$ (since 4 is the number of clauses) where G is the following graph:



The formula is satisfiable if and only if G has an independent set of size m :

- If G has an independent set S of size m , then the vertices of S must belong to different clauses. We can obtain the desired assignment of truth values to the variables by setting value $\top$ to every element of S . This assignment is consistent due to the edges joining variables with their negations. The values of the variables that have not been set by the procedure can be set arbitrarily. For the given assignment the clauses become satisfiable.
- Conversely, given a truth assignment satisfying all the clauses, we select from each clause one variable or its negation whose value is set to $\top$. The so obtained vertices form an independent set of size m in G .

The following problems are NP-complete:

- **3-SATISFIABILITY**: just like SATISFIABILITY, except that every clause consists of exactly 3 literals
(In contrast, as we already know, 2-SATISFIABILITY $\in$ P.)
- **3-COLORABILITY**: can the vertex set of a given graph be partitioned into 3 (possibly empty) independent sets?
(2-COLORABILITY $\in$ P. (A graph is 2-colorable if and only if it is bipartite.))
- **HAMILTONIAN CYCLE**: does a given graph have a Hamiltonian cycle = a cycle passing through all the vertices?

The following problems are NP-complete:

- **TRAVELING SALESMAN**: does a given edge-weighted graph have a Hamiltonian cycle of total length $\leq L$
- **METRIC TRAVELING SALESMAN**: like TRAVELING SALESMAN, except that the lengths satisfy triangle inequality
- **MAX CUT**: does a given graph admit a partition of its vertex set into two sets such that there are at least k edges between them?

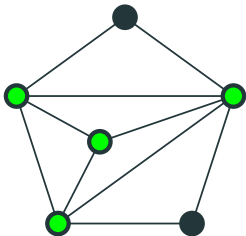
Let us now prove NP-completeness of three classical graph problems.

CLIQUE

Input: A graph $G = (V, E)$, integer k .

Question: Does G contain a clique of size $\geq k$?

clique: a subset $C \subseteq V$ such that $u, v \in C \wedge u \neq v \Rightarrow uv \in E$



Proposition

The CLIQUE problem is NP-complete.

Proof:

1. CLIQUE $\in$ NP:

for a certificate we take a clique $C \subseteq V$ with size at least k .

We can verify in polynomial time whether C is a clique and $|C| \geq k$.

2. INDEPENDENT SET $\propto$ CLIQUE

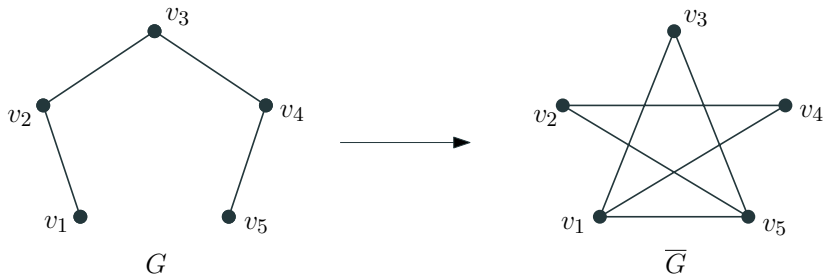
$I = (G = (V, E), k)$ instance of INDEPENDENT SET $\mapsto$

$J(I) = (\overline{G}, k)$ instance of CLIQUE

The **complement** of a graph $G = (V, E)$ is the graph $\overline{G} = (V, \binom{V}{2} \setminus E)$.

This is the graph with the same vertex set as G , in which two vertices are adjacent if and only if they are not adjacent in G .

Example:



For all $S \subseteq V$ we have: S is an independent set in $G \Leftrightarrow S$ is a clique in $\overline{G}$.

Therefore: G has an independent set of size at least k

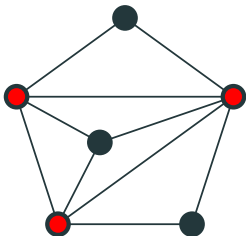
$\Leftrightarrow \overline{G}$ has a clique of size at least k .

VERTEX COVER

Input: A graph $G = (V, E)$, integer k .

Question: Does G contain a vertex cover of size $\leq k$?

vertex cover: a subset $C \subseteq V$ such that for all $e \in E$, $e \cap C \neq \emptyset$



Proposition

The VERTEX COVER problem is NP-complete.

Proof:

1. VERTEX COVER $\in$ NP.

For a certificate we take a vertex cover $C \subseteq V$ with size at most k .

We verify in polynomial time: $|C| \leq k$ and C is a vertex cover.

2. INDEPENDENT SET $\propto$ VERTEX COVER

$I = (G = (V, E), k)$ instance for INDEPENDENT SET $\mapsto$

$J(I) = (G, |V| - k)$ instance for VERTEX COVER

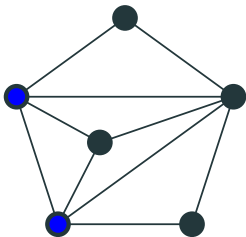
Correctness of the reduction follows from the fact that a set $S \subseteq V$ is an independent set in G with $|S| \geq k$ if and only if $V \setminus S$ is a vertex cover in G with $|V \setminus S| \leq |V| - k$.

DOMINATING SET

Input: A graph $G = (V, E)$, integer k .

Question: Does G contain a dominating set of size $\leq k$?

dominating set: a set $D \subseteq V$ such that every vertex is either in D or has a neighbor in D



Proposition

The DOMINATING SET *problem* is NP-complete.

Proof:

1. DOMINATING SET $\in$ NP:

For a certificate we take a dominating set D with size at most k . Both conditions can be verified in polynomial time.

2. VERTEX COVER $\propto$ DOMINATING SET

$I = (G = (V, E), k)$ instance for VERTEX COVER (without isolated vertices) $\mapsto$

$J(I) = (G' = (V', E'), k)$ instance for DOMINATING SET where:

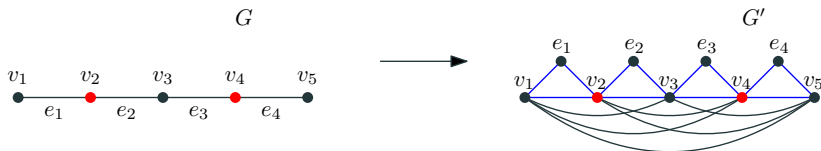
- $V' = V \cup E$,
- V is a clique in G' ,
- E is an independent set in G' ,
- $ve \in E(G') \Leftrightarrow v \in V, e \in E, v$ is an endpoint of e .



It can be shown:

G has a vertex cover of size $\leq k \Leftrightarrow$

G' has a dominating set of size $\leq k$.



The forward direction is easy: every vertex cover in G is a dominating set in G' .

For the backward direction, note that for every dominating set D in G' with $e \in D \cap E$, replacing e with one of its endpoints results in a dominating set D' in G' with $|D'| \leq |D|$.

Iteratively repeating this procedure, we obtain a dominating set in G' consisting only of vertices in V ; such a set is a vertex cover in G .

Two exercises

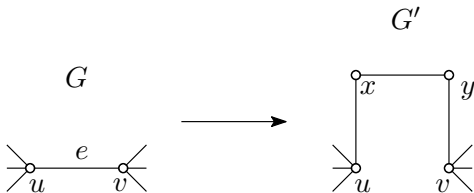
1. For a graph G let $\tau(G)$ denote the minimum size of a vertex cover in G .

(a) Show that replacing a single edge in G with a three-edge path increases the value of τ by precisely 1.

Formally: if $G = (V, E)$ is a graph and $e = uv \in E$, then

$$\tau(G') = \tau(G) + 1,$$

where $V(G') = V(G) \cup \{x, y\}$ (vertices $x, y \notin V(G)$ are two new vertices) and $E(G') = (E(G) \setminus \{e\}) \cup \{ux, xy, yv\}$.



(b) A **triangle** in a graph G is a clique of size 3.

Using the result in part (a) show that the VERTEX COVER problem is NP-hard already for graphs without triangles.

Hint: Make a reduction from VERTEX COVER.

(c) Using the result in part (b) show that the following problem is NP-complete:

TRIANGLE TRANSVERSAL

Input: A graph $G = (V, E)$, integer k .

Question: Is there a set $C \subseteq V$ that contains
at least one vertex from every triangle in G
and such that $|C| \leq k$?

2. Show that the following problem is NP-complete:

CLIQUE OR INDEPENDENT SET

Input: A graph $G = (V, E)$, integer k .

Question: Does G contain a clique of size at least k
or an independent set of size at least k ?

Hint: make a reduction from INDEPENDENT SET.

You can assume the fact that replacing a single edge in G with a three-edge path increases the independence number α by precisely 1 (that is, for the resulting graph G' we have $\alpha(G') = \alpha(G) + 1$).